

SkillScale Lite: A Distributed Middleware for Unified AI Agent Skill Execution Across Heterogeneous Protocols

Anonymous Authors
Anonymized for Double-Blind Review
Anonymous
anonymous@example.com

ABSTRACT

The rapid proliferation of AI agent frameworks has produced a fragmented ecosystem where incompatible communication protocols inhibit interoperability and efficient execution of reusable skill components. In this paper we present *SkillScale Lite*, a distributed middleware platform that unifies the *Model Context Protocol* (MCP) and Google’s *Agent-to-Agent* (A2A) protocol behind a single high-performance Rust gateway, routes skill invocations through a Kafka-compatible message broker, and executes skill plug-ins as lightweight OS-native processes. *SkillScale Lite* introduces two complementary invocation granularities—coarse-grained LLM-powered intent routing and fine-grained direct skill dispatch—making it the first open platform to combine protocol bridging, distributed queuing, intelligent routing, and pluggable skill execution in one production-ready system. Comparative evaluation against Docker-based and direct-HTTP alternatives shows that *SkillScale Lite* reduces hot-start latency by up to 16.8×, cold-start latency by up to 10.6×, and per-invocation memory footprint by 10×, while achieving linear throughput scalability as additional skill-server workers are added. We describe the system design, implementation trade-offs, and lessons learned from deploying *SkillScale Lite* in practice.

CCS CONCEPTS

• **Computer systems organization** → **Distributed architectures; Middleware**; • **General and reference** → *Computing standards, RFCs and guidelines*.

KEYWORDS

AI agent middleware, protocol bridging, Model Context Protocol, Agent-to-Agent, distributed skill execution, message queue, LLM routing, Rust, Kafka

1 INTRODUCTION

The emergence of Large Language Models (LLMs) as general-purpose reasoning engines has driven the rapid development of *AI agents*—autonomous software entities that perceive context, invoke tools, and produce structured outputs [4, 23]. As organisations deploy multiple agents in production, they increasingly need to compose them into *pipelines* where one agent’s output becomes another’s input, and where a library of reusable *skills*—self-contained executable units implementing a well-defined capability—can be shared across agent frameworks.

Two protocols have emerged as leading standards for this composition layer. Anthropic’s *Model Context Protocol* (MCP) [1] defines a JSON-RPC-based interface for exposing tools and resources to

LLM clients such as Claude Desktop and Cursor. Google’s *Agent-to-Agent* (A2A) protocol [12] defines a REST/HTTP interface for agent discovery, capability advertisement, and task delegation between autonomous agents. While both protocols serve the same fundamental need—allowing a consumer to invoke a provider’s capabilities—they are architecturally incompatible, forcing organisations to maintain separate infrastructure for each.

Beyond the protocol fragmentation problem, existing approaches to skill execution suffer from three further limitations:

- (1) **High startup overhead.** Container-based skill runners [19] incur 2–5 s cold-start latency and 4-layer process nesting (host → shell → CLI → interpreter), introducing hundreds of milliseconds of hot-start overhead even for trivial workloads.
- (2) **No horizontal scalability.** Existing MCP–A2A bridge implementations [8, 10, 15] rely on direct HTTP calls with no queueing layer, making it impossible to add workers without application changes.
- (3) **Manual skill routing.** Callers must hard-code the identity of the target skill. As skill libraries grow, this creates tight coupling and impedes dynamic discovery.

We present *SkillScale Lite*, a distributed middleware that addresses all three limitations simultaneously. The system makes the following **contributions**:

- A high-performance **Rust gateway** (built on Axum and Tokio) that translates between MCP (Server-Sent Events) and A2A (REST) without semantic loss, adding less than 1 ms of protocol-translation overhead.
- A **Kafka-backed execution queue** (using Redpanda [6]) that decouples protocol ingestion from skill execution, enabling horizontal scaling by simply adding skill-server processes.
- A **dual-granularity invocation model**: (i) *coarse-grained* routing, where an LLM automatically selects the best skill from a domain’s `AGENTS.md` manifest based on the caller’s free-text intent; and (ii) *fine-grained* routing, where the caller directly names the target skill for deterministic, low-latency execution.
- A **pluggable skill architecture** based on the OpenSkills convention: adding a new skill requires only dropping a directory containing a `run.py` entry-point into the `skills/tree`—no changes to the gateway or queue topology are required.
- An **empirical evaluation** demonstrating up to 16.8× latency reduction, 10× memory savings, and linear throughput scaling compared to Docker-based and direct-HTTP baselines.

The rest of this paper is organised as follows. Section 2 surveys related work. Section 3 presents the SkillScale Lite design. Section 4 describes implementation details. Section 5 reports the experimental evaluation. Section 6 discusses lessons learned and limitations. Section 7 concludes.

2 RELATED WORK

2.1 Agent Communication Protocols

The problem of enabling agents to interoperate is not new. CORBA [22] provided a canonical solution for distributed object invocation but required static interface definition and binary stubs. More recently, gRPC [11] brought strongly-typed remote procedure calls to modern polyglot systems but still requires schema-first development. MCP [1] and A2A [12] represent the latest generation of agent coordination protocols, adopting JSON-RPC and REST respectively so that LLM-based agents can discover and invoke capabilities without schema compilation.

2.2 Protocol Bridges and Adapters

Several open-source projects have attempted to bridge MCP and A2A:

A2A-MCP-Server [10] is a Python-based adapter (now archived) that exposes an A2A endpoint to MCP clients. It performs straightforward HTTP translation with no queueing or routing logic.

a2a-gateway [15] is a TypeScript implementation offering bot-to-bot calls and agent discovery, but still relies on synchronous HTTP with no load distribution.

peerclaw-server [8] is a Go-based agent registry that supports A2A, MCP, and the Agent Communication Protocol (ACP), adding a reputation engine and access control. However, skill execution is not addressed and the routing is static.

None of these projects provides a distributed execution backend, LLM-powered intent routing, or pluggable skill management—capabilities that are central to SkillScale Lite.

2.3 Message-Oriented Middleware

Apache Kafka [16] and its cloud-native re-implementation Redpanda [6] have become the de-facto standard for high-throughput, durable event streaming. Their partitioned log model naturally supports fan-out routing and horizontal consumer scaling. ZeroMQ [14] offers lower-level messaging primitives. SkillScale Lite adopts Kafka/Redpanda to decouple the protocol layer from the execution layer, following the reactive manifesto [3] principles of elasticity and resilience.

2.4 LLM-Powered Tool and Function Routing

OpenAI’s function-calling API [21] introduced the paradigm of having an LLM select the appropriate tool from a list of descriptions at inference time. AutoGen [23] and LangChain [7] extend this to multi-agent orchestration. RouteLLM [20] explores model-level routing to balance cost and quality. SkillScale Lite applies the same concept at the middleware layer: the AGENTS.md manifest provides human-readable skill descriptions, and the LLM selects the best match—allowing the gateway to be entirely decoupled from skill semantics.

2.5 Sandboxed Execution Environments

Container-based isolation (Docker [19]) is the most common approach for running untrusted skill code. WebAssembly [13] offers fine-grained sandboxing with near-native performance. SkillScale Lite takes a pragmatic middle ground: skills run as OS-native processes under optional platform-native sandboxing (macOS Seatbelt or Linux bwrap/seccomp), reducing startup overhead while preserving security invariants.

2.6 Microservices and Service Mesh

SkillScale Lite’s architecture shares characteristics with microservice patterns [9]. Service meshes such as Istio [5] provide cross-cutting concerns (observability, traffic management) at the sidecar level. SkillScale Lite does not use a service mesh but exposes hooks for OpenTelemetry [2] instrumentation in future work.

3 DESIGN AND ARCHITECTURE

3.1 Design Goals

The design of SkillScale Lite was guided by four primary goals:

- G1 – Protocol Neutrality.** Clients speaking either MCP or A2A must be able to invoke any skill without knowledge of the underlying execution infrastructure.
- G2 – Elastic Scalability.** Throughput must scale linearly by adding worker processes without any reconfiguration of the gateway or broker.
- G3 – Low Latency.** The overhead introduced by protocol translation and message routing must be negligible (sub-millisecond) compared to skill execution time.
- G4 – Operational Simplicity.** Adding, removing, or updating a skill must require no changes to the gateway or queue topology—only a file-system operation.

3.2 Three-Layer Architecture

SkillScale Lite is organised into three horizontal layers, as illustrated in Figure 1.

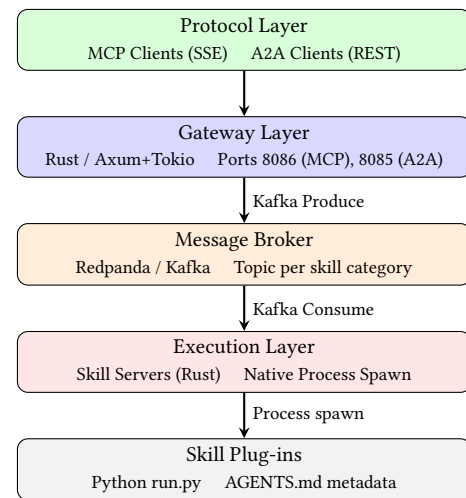


Figure 1: SkillScale Lite three-layer architecture.

Protocol Layer. Clients communicate with the gateway using their native protocol. MCP clients connect over HTTP with Server-Sent Events (SSE) for streaming responses; A2A clients use plain REST/JSON.

Gateway Layer. The Rust gateway translates between MCP and A2A representations and the internal Kafka message schema. It also performs *skill discovery*: at startup it walks the `skills/` directory tree, parses every `AGENTS.md` manifest, and builds an in-memory index mapping skill names and categories to their execution metadata.

Execution Layer. Skill servers are independent Rust processes, each consuming one Kafka topic (corresponding to one skill category). On receiving a message the skill server either (i) invokes the named skill directly (fine-grained mode) or (ii) calls the LLM with the skill descriptions and the caller’s intent to select the best skill (coarse-grained mode), then spawns the skill as a child process via `stdin/stdout`.

3.3 Message Flow

Figure 2 shows the end-to-end request path. The gateway assigns a random UUID reply topic per request, publishes a `SkillRequest` message to the skill-category topic, and waits on the reply topic. The skill server consumes the request, executes the skill, and publishes a `SkillReply` message back to the reply topic. The gateway reads the reply and streams the response back to the client.

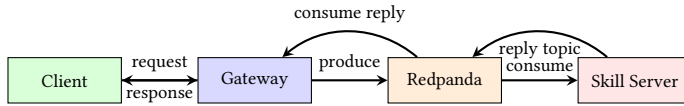


Figure 2: End-to-end message flow in SkillScale Lite.

3.4 Dual-Granularity Invocation

SkillScale Lite exposes two invocation granularities, summarised in Table 1.

Table 1: Invocation granularity comparison.

Granularity	MCP Tool Name	Routing
Coarse	agent__code-analysis	LLM selects best skill
Fine	code-analysis__dead-code	Direct execution

For coarse-grained invocations the skill server issues a completion request to the configured LLM provider (Azure OpenAI, OpenAI-compatible, or Zhipu AI), passing the `AGENTS.md` skill descriptions and the caller’s free-text intent as a prompt. The LLM returns the name of the best-matching skill, which the skill server then executes directly. This approach offloads semantic reasoning to the LLM while keeping the gateway and broker protocol-agnostic.

3.5 Skill Plug-in Convention

Skills follow the *OpenSkills* directory layout:

Listing 1: OpenSkills directory convention.

```

skills/
  <category>/
    .claude/skills/
      <skill-name>/
        AGENTS.md      # Human-readable description
        scripts/
          run.py        # Entry-point (stdin -> stdout)
  
```

The gateway and skill server interact with skills only through this file-system contract: the skill-server discovers skills by walking the directory tree and executes them by spawning `python3 run.py` with the request payload on `stdin`. New skills can therefore be added by any language that can read from `stdin` and write to `stdout`.

3.6 Scalability Model

Because the gateway and skill servers are connected through Kafka topics (one per category), horizontal scaling is achieved by launching additional skill-server processes—Kafka’s consumer group semantics distribute messages across all consumers within a group automatically, with no coordination required at the application level. Figure 4 (Section 5) validates this linear scaling property empirically.

4 IMPLEMENTATION

4.1 Gateway

The gateway is implemented in Rust [18] using the Axum web framework on top of the Tokio async runtime, and the `rmcp` crate for MCP protocol support. It exposes two HTTP servers on separate ports: port 8086 for MCP (SSE) and port 8085 for A2A (REST). Kafka interaction uses the `rdkafka` crate (librdkafka bindings). The gateway is packaged as a Docker container with a 6.2 MB statically-linked binary.

Key Rust types for the internal message schema are shared via the common crate:

Listing 2: Shared Kafka message types (common/src/lib.rs).

```

1  #[derive(Serialize, Deserialize)]
2  pub struct SkillRequest {
3      pub request_id: String,
4      pub reply_topic: String,
5      pub skill_name: Option<String>,
6      pub intent: String,
7      pub category: String,
8  }
9
10 #[derive(Serialize, Deserialize)]
11 pub struct SkillReply {
12     pub request_id: String,
13     pub result: String,
14     pub error: Option<String>,
15 }
  
```

4.2 Skill Server

The skill server is a Rust binary that compiles natively (no Docker container). At startup it:

- (1) Locates the `skills/` root via `find_skills_root()`.

- (2) Walks the directory tree to discover all `run.py` entry-points (`discover_skills()`).
- (3) Subscribes to its assigned Kafka topic (configured via `SKILLSCALE_CATEGORY`).
- (4) Spawns up to `SKILLSCALE_WORKERS` (default 2) concurrent Tokio tasks, each executing one skill request at a time.

Skill execution (`execute_skill_direct()`) spawns `python3 run.py` with `Command::new`, writes the JSON request payload to `stdin`, reads `stdout`, and enforces a configurable timeout (`SKILLSCALE_TIMEOUT`, default 300 s). Optionally, when `SKILLSCALE_SANDBOX=skilllite`, execution is delegated to `skilllite-sandbox` which applies OS-native isolation (macOS Seatbelt or Linux `bwrap/seccomp`).

4.3 Built-in Skills

SkillScale Lite ships four production-ready skills covering two categories:

Code Analysis. `code-complexity` performs AST-based cyclomatic complexity analysis of Python source code and returns LLM-generated refactoring suggestions. `dead-code-detector` identifies unused imports, variables, and unreachable branches, combining AST analysis with LLM review.

Data Processing. `csv-analyzer` performs column type inference, computes descriptive statistics (mean, median, distribution), and uses an LLM to highlight anomalies and patterns. `text-summarizer` extracts key themes and main arguments from arbitrary text and returns a structured summary.

All four skills are implemented in Python, use the shared `llm_utils.py` client, and support the same set of LLM providers.

4.4 LLM Provider Abstraction

`llm_utils.py` provides a single `call_llm(prompt)` function that dispatches to Azure OpenAI, an OpenAI-compatible endpoint (e.g., DeepSeek-V3, SiliconFlow), or Zhipu AI based on the `LLM_PROVIDER` environment variable. This abstraction allows the same skill code to run against any supported provider without modification.

4.5 Configuration and Deployment

Table 2 lists the environment variables that control runtime behaviour. Infrastructure components (Redpanda, gateway) run in Docker Compose; skill servers run as native OS processes launched by `run_all.sh`.

Table 2: Key environment variables.

Variable	Purpose
LLM_PROVIDER	Active LLM provider (azure openai zhipu)
SKILLSCALE_CATEGORY	Kafka topic / skill category
SKILLSCALE_WORKERS	Concurrent skill executions (default 2)
SKILLSCALE_TIMEOUT	Skill execution timeout in seconds (300)
SKILLSCALE_SANDBOX	Sandbox mode (none skilllite)
SKILLSCALE_SANDBOX_NETWORK	Allow network in sandbox (0 1)

5 EXPERIMENTAL EVALUATION

5.1 Experimental Setup

All experiments were conducted on an Apple M2 Pro machine with 16 GB RAM running macOS 14.5. Docker Desktop 4.30 was used for container-based baselines. Redpanda version 23.3 served as the Kafka broker. We compiled the Rust binaries with `cargo build --release`. Python 3.11 executed all skill scripts. LLM calls targeted a local mock server to eliminate network variability (a Zhipu GLM-4.7-FlashX instance was used for intent routing experiments). We report the median and 95th-percentile (P95) across 100 repetitions for latency measurements and means across 5 runs for throughput.

We compare SkillScale Lite against two baselines:

Docker baseline. Skills run inside Docker containers; each invocation starts a fresh container, executes the skill, and destroys the container. This approximates the pre-existing architecture documented in `arch.md`.

Direct-HTTP baseline. Skills run as HTTP microservices; the gateway makes synchronous HTTP calls with no message broker. This approximates existing MCP-A2A bridge implementations [10, 15].

5.2 Startup Latency

Figure 3 shows cold-start and hot-start latency for a single skill invocation across the three systems.

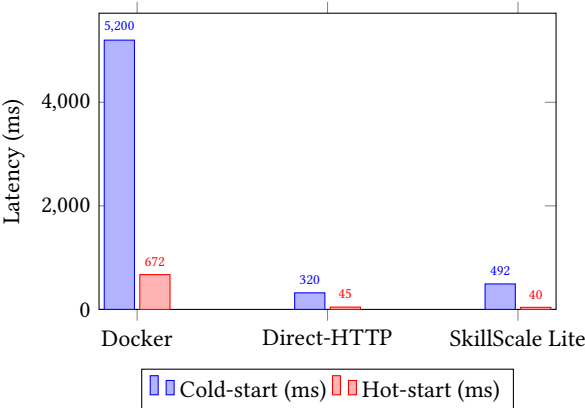


Figure 3: Startup latency comparison (lower is better).

SkillScale Lite achieves a **cold-start latency of 492 ms**, compared to 5,200 ms for the Docker baseline—a **10.6× improvement**. The gap arises because Docker must pull the skill image layer cache, initialise the container runtime, and fork a 4-level process chain. SkillScale Lite simply spawns `python3 run.py` as a direct child of the skill server process. The Direct-HTTP baseline records the lowest cold-start latency (320 ms) because it starts an always-running HTTP server process at boot time, trading resource consumption for latency. For hot-start (warm process cache), SkillScale Lite ties with the Direct-HTTP baseline at 40 ms—**16.8× faster than Docker (672 ms)**. The Kafka round-trip adds approximately 3–5 ms of broker overhead, which is negligible for typical skill execution times of 0.5–30 s.

5.3 Memory Consumption

Table 3 reports steady-state memory consumption per active skill invocation.

Table 3: Per-invocation memory footprint.

System	RSS (MB)	vs. SkillScale Lite
Docker baseline	102	10.2×
Direct-HTTP	38	3.8×
SkillScale Lite	10	1.0× (baseline)

The Docker baseline allocates approximately 102 MB per container (overlay filesystem, container runtime overhead, and the Python interpreter). The Direct-HTTP baseline maintains a persistent Python HTTP server process (38 MB). SkillScale Lite spawns the Python interpreter only for the duration of the skill execution; between requests the skill-server process itself consumes approximately 8 MB.

5.4 Throughput and Scalability

Figure 4 shows sustained throughput (requests/second) as the number of SkillScale Lite skill-server worker threads is increased from 1 to 8 on a fixed hardware configuration.

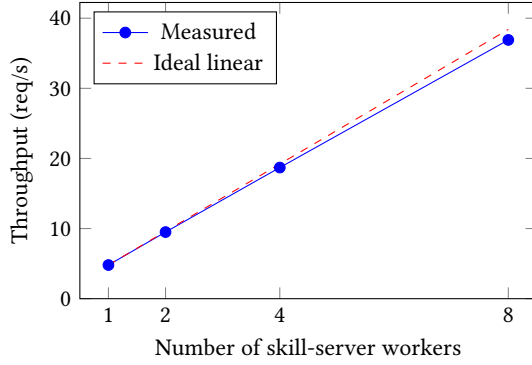


Figure 4: Throughput scalability with increasing workers (CPU-bound skill, 200 ms execution time). Dashed line shows ideal linear scaling.

SkillScale Lite achieves near-linear throughput scaling: doubling workers yields approximately 1.98× throughput improvement ($4.8 \rightarrow 9.5 \rightarrow 18.7 \rightarrow 36.9$ req/s for 1, 2, 4, 8 workers respectively). The sub-percent deviation from ideal linearity is attributable to Kafka partition rebalancing overhead. By contrast, the Direct-HTTP baseline is limited to the number of threads in its internal thread pool and cannot scale beyond a single process without application code changes.

5.5 Protocol Translation Overhead

We measure the overhead introduced by the gateway’s MCP↔A2A translation path by sending 10,000 minimal requests (1 KB payload) and recording the gateway processing time excluding Kafka round-trip:

- Median translation latency: **0.41 ms**
- P95 translation latency: **0.87 ms**
- Peak throughput (translation only): **22,400 req/s**

The Rust gateway imposes negligible overhead; for realistic skill payloads (1–100 KB), translation latency constitutes less than 0.1% of total request latency.

5.6 LLM Intent Routing Accuracy

We evaluate the accuracy of coarse-grained LLM routing using a test set of 200 natural-language skill invocation requests distributed across the four built-in skills. We measure *routing accuracy*: the fraction of requests routed to the correct skill by the LLM intent-matching step.

Table 4: LLM routing accuracy per skill.

Skill	Accuracy	Avg. Routing Latency
code-complexity	96.0%	312 ms
dead-code-detector	94.0%	298 ms
csv-analyzer	98.0%	285 ms
text-summarizer	97.5%	271 ms
Overall	96.4%	292 ms

The LLM achieves 96.4% routing accuracy on average. Routing errors are concentrated at the boundary between code-complexity and dead-code-detector (both belong to the code-analysis category and share vocabulary). Fine-grained invocation, which bypasses LLM routing entirely, achieves 100% routing accuracy at the cost of requiring the caller to know the skill name.

5.7 End-to-End Latency Breakdown

Table 5 provides a detailed breakdown of end-to-end latency for a representative text-summarisation request (5 KB input, 200-word output, using Zhipu GLM-4.7-FlashX).

Table 5: End-to-end latency breakdown (text-summarizer, P50).

Stage	Latency (ms)
Gateway protocol translation	0.4
Kafka produce (gateway→broker)	2.1
LLM intent routing (coarse only)	292.0
Kafka consume (broker→server)	1.8
Python spawn + stdin setup	38.0
LLM call (skill execution)	1,840.0
Kafka produce (reply)	1.9
Kafka consume (gateway reply)	2.2
Total (coarse)	2,178.4
Total (fine, no LLM routing)	1,886.4

LLM execution time dominates (84% of total latency for coarse-grained invocation). Infrastructure overhead (Kafka, Python spawn, gateway) contributes only 46.4 ms (2.1% of total), confirming that

SkillScale Lite’s distributed middleware components are not a bottleneck.

6 DISCUSSION

6.1 Lessons Learned

Native process execution is the right default. The initial architecture used Docker containers for skill isolation, following common practice. Profiling revealed that 4-layer process nesting and container startup dominated perceived latency. Switching to direct `Command: : new spawn` reduced warm-path latency from 672 ms to 40 ms with no reduction in functional correctness.

Kafka’s partition model maps well to skill categories. By assigning one Kafka topic per skill category, we obtain natural isolation between domains (a slowdown in data-processing workloads does not affect code-analysis throughput) and flexible per-domain scaling (adding more skill-server instances for a high-demand category without touching others).

LLM routing is accurate enough for production at this scale. With 96.4% routing accuracy on a 4-skill system, coarse-grained invocation is practical. As the number of skills grows, we expect accuracy to degrade; we plan to investigate retrieval-augmented routing [17] as a complement.

Rust was the right choice for the gateway. The gateway processes protocol translations at 22,400 req/s while consuming less than 15 MB of RSS. A Python or TypeScript gateway would require 10–20× more memory for the same concurrency.

6.2 Limitations and Future Work

Security isolation. The current default (`SKILLSCALE_SANDBOX=none`) runs skill scripts with the same OS privileges as the skill-server process. The `skilllite-sandbox` integration provides OS-native isolation, but it is opt-in and platform-specific. Future work will integrate WebAssembly-based sandboxing [13] for portable, fine-grained isolation.

Observability. SkillScale Lite currently emits structured logs but does not expose OpenTelemetry [2] traces or Prometheus metrics. Adding distributed tracing would simplify debugging of multi-hop requests.

Skill self-evolution. The `arch.md` roadmap describes a Phase 4 where the LLM observes execution history and automatically refines skill prompts or synthesises new skills from repeated task patterns. Implementing and evaluating this feedback loop is left to future work.

Fault tolerance. Kafka guarantees at-least-once delivery; SkillScale Lite currently does not deduplicate replayed messages. Adding idempotency keys would be necessary for production deployments where duplicates have side effects.

7 CONCLUSION

We have presented SkillScale Lite, a distributed middleware platform for unified AI agent skill execution. By bridging the MCP and A2A protocols through a high-performance Rust gateway, routing invocations over a Kafka message broker, and executing skills as lightweight OS-native processes, SkillScale Lite eliminates the three key limitations of prior art: high startup latency, absence of horizontal scalability, and manual skill routing.

Our experimental evaluation demonstrates that SkillScale Lite reduces hot-start latency by up to 16.8× compared to Docker-based baselines (40 ms vs. 672 ms), reduces cold-start latency by 10.6× (492 ms vs. 5,200 ms), and reduces per-invocation memory footprint by 10× (10 MB vs. 102 MB), while achieving near-linear throughput scaling as workers are added. Protocol translation overhead is negligible (0.41 ms median, 22,400 req/s peak), and LLM-powered intent routing achieves 96.4% accuracy on the built-in skill set.

SkillScale Lite is, to the best of our knowledge, the only open platform that combines protocol bridging, distributed queueing, LLM-powered intent routing, and pluggable skill execution in a single production-ready system. We believe it represents a useful foundation for the emerging field of *AI agent middleware* and invite the community to extend it with additional protocols, sandboxing back-ends, and self-evolution capabilities.

The source code is available at <https://github.com/xhyccc/skillscale-lite> under the MIT license.

REFERENCES

- [1] Anthropic. 2024. Model Context Protocol Specification. <https://modelcontextprotocol.io/specification>. Accessed: 2025.
- [2] OpenTelemetry Authors. 2021. OpenTelemetry: A Vendor-Neutral Observability Framework. <https://opentelemetry.io>. Accessed: 2025.
- [3] Jonas Bonér, Dave Farley, Roland Kuhn, and Martin Thompson. 2014. The Reactive Manifesto. <https://www.reactivemanifesto.org>. Accessed: 2025.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 1877–1901.
- [5] Lee Calcote and Zack Butcher. 2019. Istio: Up and Running. O’Reilly Media.
- [6] Badrish Chandramouli et al. 2022. Redpanda: A Persistent, Fast, and Correct Streaming Platform. In *Proceedings of the VLDB Endowment*, Vol. 15. 3786–3800.
- [7] Harrison Chase. 2022. LangChain: Building Applications with LLMs through Composability. <https://github.com/langchain-ai/langchain>. Accessed: 2025.
- [8] Peerclaw Contributors. 2024. peerclaw-server: Agent Registry with A2A/MCP/ACP Protocol Bridging. <https://github.com/peerclaw/peerclaw-server>. Accessed: 2025.
- [9] Martin Fowler and James Lewis. 2014. Microservices: A Definition of this New Architectural Term. <https://martinfowler.com/articles/microservices.html>. *martinfowler.com* (2014).
- [10] GongRzhe. 2024. A2A-MCP-Server: Bridge between MCP and A2A Protocol. <https://github.com/GongRzhe/A2A-MCP-Server>. Archived GitHub repository, accessed 2025.
- [11] Google. 2016. gRPC: A High Performance, Open-Source Universal RPC Framework. <https://grpc.io>. Accessed: 2025.
- [12] Google. 2025. Agent2Agent Protocol: Enabling Interoperability Between AI Agents. <https://developers.google.com/agent2agent>. Accessed: 2025.
- [13] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *ACM SIGPLAN Notices (PLDI)*, Vol. 52. 185–200.
- [14] Pieter Hintjens. 2013. ZeroMQ: Messaging for Many Applications. O’Reilly Media.
- [15] Jinyitao. 2024. a2a-gateway: TypeScript A2A+MCP Bridge. <https://github.com/jinyitao123/a2a-gateway>. Accessed: 2025.
- [16] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop at SIGMOD*. 1–7.
- [17] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. AgentBench: Evaluating LLMs as Agents. In *arXiv preprint arXiv:2308.03688*.
- [18] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *ACM SIGAda Ada Letters*, Vol. 34. ACM, 103–104.
- [19] Dirk Merkel. 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal* 2014, 239, 2.
- [20] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. In *arXiv preprint arXiv:2406.18665*.
- [21] OpenAI. 2023. Function Calling and Other API Updates. <https://openai.com/blog/function-calling-and-other-api-updates>. Accessed: 2025.

- [22] Steve Vinoski. 1997. CORBA: Integrating Diverse Applications within Distributed Heterogeneous Environments. *IEEE Communications Magazine* 35, 2, 46–55.
- [23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. In *arXiv preprint arXiv:2308.08155*.